***"Arquitectura Inicial - CLE"***

Vega Ángeles Christopher

22140787

Moya Arreola Cristian Moya

22140757

Ruiz Avilés Fernanda Michelle

22140756

Casas Vázquez Luis Américo

22140750

*Ingeniería en Sistemas Computacionales, Instituto Tecnológico de Querétaro*.

Taller de Desarrollo Ágil de Software

José Miguel Jaime Ramírez

Índice

Visión General del Sistema ..... 3
API Gateway, Red y Exposición ..... 3
Microservicios Iniciales y Responsabilidades ..... 4
Microservicio de Autenticación (usuarios-servicio) ..... 4
Microservicio de Gestión Académica (academico-servicio) ..... 4
Microservicio de Operación e Infraestructura ..... 4
Capa de Bases de Datos ..... 4
Comunicación y Flujo de Integridad (Inter-servicios) ..... 5

# Visión General del Sistema

La plataforma CLE está construida bajo una arquitectura de microservicios orientada al dominio, diseñada para garantizar alta concurrencia, escalabilidad y tolerancia a fallos. El sistema se divide en dos bloques principales:

**Frontend (Cliente):** Una Single Page Application (SPA) reactiva desarrollada con React, Vite y Tailwind CSS, encargada de la interfaz de usuario (roles: Administrador, Maestro, Alumno).

**Backend (Servidor):** Un conjunto de microservicios desarrollados en Node.js y Express, contenerizados con Docker y orquestados para manejar la lógica de negocio.

# API Gateway, Red y Exposición

La comunicación entre el mundo exterior y los microservicios está fuertemente controlada mediante capas de red:

**Cloudflare Tunnel y Proxy Vite:** Se encargan de la exposición segura de la plataforma hacia internet y del enrutamiento inicial.

**Nginx (API Gateway):** Actúa como el proxy inverso principal dentro de la instancia de AWS EC2. Recibe las peticiones HTTP seguras y las distribuye al contenedor Docker del microservicio correspondiente. Esto oculta la topología de la red interna y centraliza el manejo del tráfico.

# Microservicios Iniciales y Responsabilidades

Para aislar las funcionalidades, el backend opera con 3 microservicios independientes principales:

### Microservicio de Autenticación (usuarios-servicio)

**Responsabilidades:** Es el guardián del sistema (RBAC - Control de Acceso Basado en Roles). Gestiona el inicio de sesión, la creación de cuentas, la encriptación de contraseñas y la emisión de tokens JWT (Bearer Auth). Bloquea o permite el acceso a las rutas protegidas según el perfil del usuario.

**Base de datos asignada:** cle_usuarios (Tablas: usuario, rol, estatus_usuario).

### Microservicio de Gestión Académica (academico-servicio)

**Responsabilidades:** Centraliza la lógica educativa. Administra el catálogo de idiomas, los niveles, las unidades de evaluación y los expedientes de alumnos (internos/externos) y maestros (incluyendo sus certificaciones).

**Base de datos asignada:** cle_academico (9 tablas relacionales).

### Microservicio de Operación e Infraestructura

**Responsabilidades:** Maneja la logística en tiempo real. Gestiona la apertura de periodos escolares, creación de grupos, programación de horarios sin traslapes, asignación de campus/salones y el procesamiento de inscripciones. Realiza el monitoreo analítico de cupos críticos y demanda.

**Bases de datos asignadas:** cle_infraestructura (2 tablas) y cle_inscripciones (5 tablas).

# Capa de Bases de Datos

Se emplea el patrón arquitectónico *Database-per-Service* para mantener la independencia de los microservicios.

- **Infraestructura Cloud:** La persistencia de datos recae en el motor **PostgreSQL**, alojado en una única instancia administrada de **Amazon RDS** (cle-postgresql).

- **Segmentación Lógica:** Dentro de la instancia de RDS, coexisten 4 bases de datos lógicas independientes. En total, la arquitectura inicial contempla 19 tablas estructuradas de forma relacional.

# Comunicación y Flujo de Integridad (Inter-servicios)

**Comunicación Externa (Cliente-API):** El frontend consume los microservicios de forma asíncrona mediante peticiones HTTP RESTful usando la librería Axios. Cada petición va firmada con un token JWT validado por interceptores de sesión.

**Flujo Interno por Capas:** Cada microservicio procesa las peticiones a través de un diseño de capas estrictas: Routes (con validación Zod) → Controllers → Services → Repositories → BD.

**Integridad Referencial:** Dado que las 4 bases de datos están separadas, **no existen llaves foráneas físicas entre ellas**. La comunicación e integridad referencial cruzada (por ejemplo, inscribir un alumno_id en un grupo_id) se maneja mediante referencias lógicas. Es la capa de *Services* de cada microservicio la responsable de validar la existencia de estos registros antes de concretar una operación en la base de datos.

# Bibliografía

**Amazon Web Services.** (2026). *Documentación oficial de AWS (Amazon EC2 y Amazon RDS)*. Recuperado de: https://docs.aws.amazon.com/es_es/

**Docker Inc.** (2026). *Docker Documentation: Guías y manuales de referencia*. Recuperado de: https://docs.docker.com/

**JSON Web Token.** (2026). *Introduction to JSON Web Tokens*. Recuperado de: https://jwt.io/introduction

**PostgreSQL Global Development Group.** (2026). *Manual oficial de PostgreSQL*. Recuperado de: https://www.postgresql.org/docs/